here's a **plain-English, line-by-line explanation** of your deterministic **GAO-Spiral** pseudocode.

GAO-Spiral(P)

Goal:
Given a set of points PP, this algorithm builds an ordered tour that spirals outward from the center, balancing angular and radial coherence.

```
C ← centroid(P)
```

- Compute the **center point** (average x and y) of all points.
- This will serve as the spiral's **origin**.

```
for p in P: (r[p], θ[p]) ← polar(p - C)
```

- Convert every point's coordinates into **polar form** (radius rr, angle θθ) relative to the centroid.
- So each point now has:
  - r[p]r[p]: distance from center
  - θ[p]θ[p]: angle around the center

```
α ← 0; visited ← ∅
```

- Initialize:
  - `α` = current **target angle**, starting at 0 radians (the x-axis direction)
  - `visited` = an empty set to track which points have already been added to the tour

```
current ← argmin_p r[p]
```

- Choose the **innermost point** (smallest radius) as the **starting point**.
- The spiral will grow outward from this seed.

Main loop

```
while |visited| < n:
```

- Continue until all points have been visited.

```
add current to tour; visited ← visited ∪ {current}
```

- Record the current point in the tour sequence.
- Mark it as visited so it won't be chosen again.

```
α ← (α + φ) mod 2π
```

- Move the target angle **forward by a small step φ** (the spiral's angular increment).
- Wrap around when it passes 2π2π (a full circle).
- This creates the **spiral rotation** pattern.

Angular bin lookup

```
S ← BIN_NEAR_ALPHA(α) \ visited
```

- Find the set `S` of **unvisited points** whose angle θ[p]θ[p] lies **near** the current target angle αα.
- The bin width `h` (angular tolerance) can scale with the number of points (between O(1/n)O(1/n) and O(1/√n)O(1/√n)).
- In other words: look for nearby points in direction `α`.

```
if S empty: S ← NEXT_NONEMPTY_BIN()
```

- If there are no unvisited points near that angle, **jump to the next angular sector** that still has points left.
- Prevents the spiral from getting stuck when some regions are already cleared.

Local selection rule

```
next ← argmin_{p ∈ S} [ wθ·Δθ(p|α) + wr·|r[p]-r[current]|/r̄ ]
```

- Among candidates in `S`, pick the **next point** that best balances:
    1. **Angular alignment** — how close its angle θ[p]θ[p] is to the target direction αα
    2. **Radial smoothness** — how similar its radius is to the current point's radius (scaled by average radius r̄r̄)
- The weights `wθ` and `wr` tune the emphasis on angular vs. radial coherence.

```
current ← next
```

- Move to that next point and continue the spiral.

End of loop

```
return tour + [tour[0]]
```

- Once all points are visited, return the **completed tour**, closing the loop by returning to the start.

In summary

This algorithm:

1. Finds the center of all points.
2. Starts from the innermost point.
3. Spirals outward, sweeping around in small angular increments.
4. At each step, chooses the next point that’s near the current angle and at a similar distance from the center.
5. Produces a smooth, outward-winding tour that maintains both angular order and radial continuity — a “geometric coherence spiral.”